Weekly Project Progress Report

Team Name: Quick Bite

Week Number: Week 10 – Server-side implementation

Team Members:

1. Abdullah Mohammed AbuZaid 120211559
2. Abdullah Ali Al-Hindawi 120211863
3. Abdalkareem Rajab Abo Younis 120211877
4. Hazem Mohamed Oukal 120212771

GitHub Repository: https://github.com/AbdullahAbuZaid04/quick-bite-restaurant.git

Work completed this week

This week the backend team finished and delivered the server-side implementation for the Quick Bite system. We built a Node.js + Express REST API on top of MySQL, organised as a layered architecture (routes → middleware → controllers → database) so each concern lives in its own module. Hazem owned the project foundation, database design, shared infrastructure (configuration, error handling, validation, authentication), and the identity and catalog subsystems (Auth, Users, Categories, Menu). Abdalkareem owned the business-logic layer: order processing with transactional integrity, payment handling, automatic invoice generation, and the admin dashboard. The work was coordinated through two separate feature branches and two pull requests on GitHub, both of which were merged into main by the end of the week.

Features implemented

1. Authentication using JWT tokens with bcrypt password hashing, and role-based authorization for four roles (customer, admin, manager, courier).
2. Full CRUD endpoints for categories and menu items, including search/filter and a quick availability toggle for admins.
3. Transactional order creation that reads prices from the database (never from the client) and uses SELECT … FOR UPDATE row locks to prevent race conditions on concurrent orders.
4. An explicit order-status state-machine: pending → confirmed → preparing → out_for_delivery → delivered, with cancel and refund branches.
5. Payment endpoints whose status transition triggers an atomic cascade: on payment marked as paid, the order is auto-confirmed and an invoice is auto-issued in the same transaction.
6. An admin dashboard endpoint that aggregates ten summary queries in parallel for fast page loads (user counts, orders by status, revenue today and total, recent orders).

7. A zero-dependency smoke test (17 assertions, all passing) that exercises routing, validation, JWT auth, and role gating without requiring a live MySQL instance.
8. Security defaults: helmet HTTP headers, CORS allow-list, global rate limiting (300 req/min/IP), tighter limit on the login endpoint (20 attempts per 15 min/IP), and parameterized SQL on every query.

Problems encountered

1. Concurrent orders for the same menu item created a time-of-check-to-time-of-use gap between reading the menu price and inserting the order line item, which could allow a stale price to leak into a new order.
2. Raw MySQL error codes such as ER_DUP_ENTRY, ER_NO_REFERENCED_ROW_2, and ER_ROW_IS_REFERENCED_2 were leaking into API responses and were not human-readable.
3. Validation schemas were initially duplicated across controllers, which caused rules to drift as we made changes.
4. Updating a payment's status needed to trigger two side-effects (auto-confirm the order and auto-issue the invoice) without leaving the system in an inconsistent state if any step failed.
5. Two developers committing to the same backend folder during the same week risked merge conflicts on shared files such as the routes aggregator.

Solutions applied

1. Order creation now runs inside a single MySQL transaction: getConnection → beginTransaction → SELECT … FOR UPDATE → INSERT → commit/rollback. The menu rows are row-locked for the duration of the transaction, closing the price-leak window, and a database trigger keeps the order total consistent with the line items.
2. A centralized Express error-handler middleware translates the known MySQL error codes into clean 4xx responses with friendly messages, so the API never leaks raw database errors.
3. All Joi schemas were lifted into a single src/validators/ folder and applied through one generic validate(schema, source) middleware, giving each validation rule exactly one definition.
4. The payment cascade (confirm order plus issue invoice) is wrapped in the same database transaction as the payment update itself, so either every step succeeds or nothing changes.
5. We adopted a branch-per-developer workflow with pull requests: Hazem worked on feature/backend-auth-catalog, Abdalkareem on feature/backend-orders-payments, and the file split was non-overlapping by design (different controllers, validators, and routes) so both PRs merged into main with zero conflicts.

Plan for next week

1. Add an image-upload endpoint for menu items, using multer to store images either on local disk or an S3-compatible bucket.

2. Add server-side pagination, sorting, and filtering to the orders list endpoint so admin pages stay fast as the order history grows.
3. Implement a refund flow that pairs the payment status transition paid → refunded with an order transition delivered → refunded.
4. Begin Phase 4 work: containerize the service with Docker and prepare a deployment target (Render or Railway) with a managed MySQL instance.

Questions for instructor

1. For the final submission, would you prefer the database SQL dump (schema plus sample data) committed to the repository, or only the schema file?
2. For deployment, is a free hosted PaaS such as Render or Railway acceptable, or should we plan for a local-only setup that you can run from the README during evaluation?
3. Are integration tests against a real test database required, or is our current process-level smoke test sufficient for this phase?